Finite Automata in Haskell

V. Velkey, W. Vromen

Sunday 4th June, 2023

Abstract

Finite state automata are popular tools to analyse formal languages. This project aims to implement finite state automata, both deterministic and nondeterministic. We also implement regular expressions and generate words from their language. Finally, we implement translations between our objects and run several tests to verify our implementation.

Contents

1 Introduction

Finite state automata are mathematical models of computation. They are mostly used to study regular languages, i.e. languages with a certain structure. This structure is also captured by regular expressions.
In this report we discuss our Haskell-implementation of finite state automata and regular expressions. In particular, we implemented deterministic finite state automata, non-deterministic finite state automata and regular expressions. The goal is to make the equivalence of finite state automata and regular languages explicit by various translation functions. Regular expressions are suitable for generating words from a regular language, but finite automata are more suitable for determining whether a word is in its language. Our implementation enables combining these features. For example, given a regular expression, we can compute another one that generates the complement language using finite state automata. Moreover, given an automaton, we can generate words that it accepts using regular expressions. Furthermore, we provide a toolkit that determines the minimal equivalent DFA to a DFA and checks isomorphism of DFAs.
Our report is structured as follows. In Section 2 we give a brief introduction to the theory of regular languages and point towards more detailed references. In Sections 3 - 6 we discuss our implementation of finite state automata and regular expressions. In Section 7 we turn to the implementation of the translation functions. We then provide extensive testing apparatus for our implementation in Section 8 and conclude in Section 9.

2 Background Theory

Automata are mathematical models of computation. They are used in several areas of computer science and linguistics. Finite state automata, one of the most basic models, are used in, among other things, text processing and compilers. Here we will give a brief overview of the theory, which should be enough to make one understand the report. For a more detailed exposition, see for example [Sip13].

2.1 Formal Languages

Automata are concerned with *formal languages*. A language in our setting is nothing more than a set of strings, or *words*, constructed from a given (finite) *alphabet*. An alphabet is typically denoted by the Greek capital Σ and consists of *symbols* or characters. A string over an alphabet is just a finite sequence of symbols. If A is a set, then the set of all finite sequences over A is denoted A^*. Hence A^* is the full language over alphabet A.
One special symbol that is a member of A^* for any A is ϵ. This character denotes the *empty string*, or the word of length 0.
An example of an alphabet is $\Sigma = \{0, 1\}$ and one of the many possible languages over Σ is $B = \{0w \mid w \in \Sigma^*\}$. This language contains all binary strings starting with 0 and nothing more.

2.2 Combining Languages and Regular Expressions

We define several operations over languages to construct new ones. These are called the *regular operations*.
Let A and B be formal languages over alphabet Σ. Then:

- Their union, denoted as $A \cup B$, is defined as $\{w \mid w \in A \cup B\}$.
- Their concatenation, denoted as $A \circ B$ or AB, is defined as $\{wv \mid w \in A,\ v \in B\}$.

- We have already seen the closure operator *, called the *Kleene star*, to get $A^* := \{w_1 w_2 \dots w_n \mid \forall i : w_i \in A,\ n \in \mathbb{N}\}$.
- Finally, we also have $A^+ := A^* \setminus \{\epsilon\}$ to get all non-empty sequences made with words from A. Observe that A^+ is equivalent to AA^*.

These operations naturally give rise to *regular expressions*. A regular expression r always corresponds to a language, $\mathcal{L}(r)$.

Definition 2.1 (Regular Expressions)**.** Let Σ be an alphabet. The set $\mathcal{R}$ of regular expressions over Σ are recursively defined as follows:

- $\emptyset \in \mathcal{R}$, denoting the empty language: $\mathcal{L}(\emptyset) = \emptyset$.
- $\epsilon \in \mathcal{R}$, denoting $\mathcal{L}(\epsilon) = \{\epsilon\}$.
- $a \in \mathcal{R}$, with $\mathcal{L}(a) = \{a\}$ for all $a \in \Sigma$.

If $r, r' \in \mathcal{R}$, then:

- $r + r' \in R$, also written as $r|r'$ or $r \cup r'$, corresponding to $\mathcal{L}(r) \cup \mathcal{L}(r')$.
- $rr' \in \mathcal{R}$ with $\mathcal{L}(rr') = \mathcal{L}(r) \circ \mathcal{L}(r')$.
- r^* corresponding to $\mathcal{L}(r)^*$.
- r^+ with $\mathcal{L}(r^+)^+ = \mathcal{L}(r)^+$.

A regular expression r is like a pattern; any word that matches the pattern is *generated by* r. Languages generated by regular expressions are called *regular languages*.

2.3 Finite State Automata

As stated earlier, a finite state automaton (FSA) is a model of computation. Specifically, an FSA operates on words. When inputting a word into an FSA, it will either accept or reject. The set of all words that are accepted by an FSA A is called the language of A, or the language recognized by A. It is denoted $\mathcal{L}(A)$. We distinguish deterministic finite state automata (DFAs) from non-deterministic finite state automata ($(\epsilon$-)NFAs). The formal definition of a DFA is as follows:

Definition 2.2. A DFA D is a 5-tuple $(S, \Sigma, \delta, s_0, F)$, where:

- S is a non-empty set of *states*.
- Σ is the alphabet on which D operates.
- $\delta : S \times \Sigma \to S$ is the transition function[1]
- $s_0 \in S$ is the starting state.
- $F \subseteq S$ is the set of accepting states.

The computation of a word w on DFA $D = (S, \Sigma, \delta, s_0, F)$ goes as follows:

1. Set the current state s to be s_0.

[1]When considering an NFA, the only difference is that this is a transition relation $\Delta \subseteq S \times \Sigma \times S$.

2.
   - If $w = \epsilon$, accept if $s \in F$ and reject otherwise.
   - If $w = aw'$ where $a \in \Sigma$, set $s := \delta(s, a)$ to be the next state and set $w := w'$. Repeat step 2.

We have three remarks about this procedure:

- When reading a word, it is up for debate if a DFA rejects the word or fails entirely when it reads a symbol that is not in the alphabet; our implementation will reject for a more user-friendly experience.
- An NFA could end up in multiple states when reading a symbol. The idea is that the machine will non-deterministically choose to which state to go. If one possible path of computation on a word (also known as a *run* of w) ends up in an accepting state, the machine will accept. The run is then called *successful*. Note that some paths may end due to the machine reading a symbol a in some state s such that there is no t with $(s, a, t) \in \Delta$. In that case this path will count as not successful.
- An ϵ-NFA extends the notion of an NFA by allowing for transitioning states on the empty symbol, ϵ. Before reading a symbol, the currently visited states need to be updated by all possible ϵ-transitions. This is done by taking the reflexive and transitive closure of the ϵ-relation of states.

Two FSAs are called *equivalent* just in case they recognize the same language. It may not be surprising that for every DFA there is an equivalent NFA: every function is a relation after all. However, it turns out that for every NFA there is also an equivalent DFA. Moreover, every language that is accepted by an FSA is also generated by some regular expression and vice versa. I.e. the languages recognized by FSAs are exactly the regular ones.
In the following sections we will discuss how we implemented all these notions, and how our program shows the facts stated above and more.

3 DFA Implementation

This section describes our implementation of DFAs, together with some helpful functions to work with them.

3.1 Data type and basics

As in the formal definition, a DFA consists of a tuple $(Q, \Sigma, \delta, q_{start}, A)$ representing the set of states, the alphabet, the transition function, the start state and the accept states. Throughout, we chose to represent sets as lists, as working with lists is generally easier and faster.
The parameter of the DFA constructor stands for what type the states are. By default, we will use DFAs with integer states, but occasionally we will need ones that have lists of integers as their states.

```
{-# LANGUAGE FlexibleInstances #-}
module DFA where
import Data.List ( (\\), intersect, nub )
import Test.QuickCheck ( elements, sublistOf, Arbitrary(arbitrary), Gen )

type Symbol = Char
data DFA a = DFA { states :: [a]
                 , alphabet:: [Symbol]
                 , delta :: Symbol -> a -> a
                 , start:: a
                 , acceptstate:: [a]}
```

We implemented an evaluation function that, given a string, determines if the string is in the language. For this, we use a helper function `stateArr` that specifies the current state after reading a list of symbols.

```
evaluate:: (Eq a, Ord a) => DFA a -> String -> Bool
evaluate df st = all (`elem` alphabet df) st && -- captures that all element of string in
    alphabet
                stateArr (start df) st `elem` acceptstate df where
                    stateArr  q [] = q
                    stateArr q (x:xs) = stateArr (delta df x q) xs
```

Below is an example DFA on the alphabet $\Sigma = \{0, 1\}$, accepting the language of words starting with a 0.

```
zeroStart:: DFA Int
zeroStart = DFA [0,1,2] ['0', '1'] deltazero 0 [1] where
    deltazero:: Symbol -> Int -> Int
    deltazero char st
     | st == 0 && char == '0' = 1
     | st == 0 && char == '1' = 2
     | st == 1 = 1
     | st == 2 = 2
     | otherwise = -1
```

We add a basic `show` instance for DFAs. The transition function δ is shown as a list of strings specifying its value on the state space and the alphabet.

```
instance Show a => Show (DFA a) where
    show (DFA sts alph del strt acc)
      = "DFA" ++ "("++ show sts ++ "," ++ show alph ++ "," ++ show1 del ++ "," ++ show strt ++
          "," ++ show acc ++")" where
      show1 f = show ["d" ++ "(" ++ show sym ++ "," ++ show st ++ ")" ++ " = " ++ show (f sym
          st) | sym <- alph, st <- sts ]
```

On the DFA zeroStart from above, this yields the following string representation.

```
ghci> zeroStart
DFA([0,1,2],"01",["d('0',0) = 1","d('0',1) = 1","d('0',2) = 2","d('1',0) = 2","d('1',1) = 1","
    d('1',2) = 2"],0,[1])
```

3.2 Helpful functions

Below we defined some helpful functions.
The `flipDFA` function takes a DFA and outputs that DFA, but with its accepting states now rejecting and vice versa. The `cutDFA` function creates a DFA whose state space consists of just those states that are reachable from the start state via the δ-transitions. The `reachables` function finds those reachable states. Using the `reachables`, we wrote a function that determines whether a given DFA has an empty language.

```
flipDFA :: (Eq a, Ord a) => DFA a -> DFA a
flipDFA (DFA sts alp del st acc) = DFA sts alp del st (sts \\ acc)

reachables :: Eq a => DFA a -> [a]
reachables dfa = reachableInSteps [start dfa] where
  reachableInSteps ts | all (`elem` ts) (concatMap allSuccessors ts) = ts -- if all successors
       are in the set then
  --no more reachables
                      | otherwise = reachableInSteps (nub $ ts ++ concatMap allSuccessors ts)
                          --else add successors
  allSuccessors st = nub (st : [delta dfa sym st | sym <- alphabet dfa])

cutDFA :: Eq a => DFA a -> DFA a
cutDFA d = DFA sts alp delt strt acc where
  sts = reachables d
  alp = alphabet d
  strt = start d
  acc = reachables d `intersect` acceptstate d
  delt = delta d
```

```
languageIsEmpty :: Eq a => DFA a -> Bool
languageIsEmpty d = null $ reachables d `intersect` acceptstate d
```

3.3 Arbitrary generation

For testing purposes, we added the functionality to arbitrarily generate DFAs. For this we make DFAs an instance of the `Arbitrary` class. To generate arbitrary delta functions, we use two helper functions. The first generates arbitrary functions on the state space, the second adds the `Symbol` argument to the function. In the construction, we make sure that the state space and set of accept states are non-empty, by adding 0 to both. For running time purposes, we only generate DFAs that have at most 4 states, but that can be adjusted as described in the code.

```
-- inspired by arbitrary instance in HW2
randomFunFromTo :: (Eq a, Ord a, Arbitrary a) => [a] -> [a] -> Gen (a -> a)
randomFunFromTo [] _ = return (const undefined)
randomFunFromTo (w:ws) ps = do
    f <- randomFunFromTo ws ps
    wResult <- elements ps
    return $ \v -> if v == w then wResult else f v

randomDelta :: (Eq a, Ord a, Arbitrary a) => [Symbol] -> [a] -> [a] -> Gen (Symbol -> a -> a)
randomDelta [] _ _ = return (const undefined)
randomDelta (sym:syms) ds ps = do
    f <- randomDelta syms ds ps
    wResult <- randomFunFromTo ds ps
    return $ \sy -> if sy == sym then wResult else f sy

instance Arbitrary (DFA Int) where
    arbitrary = do
        sts <- (0 :) <$> sublistOf [1..3] --choose a set up to 4 states. to change this, set 3
             to some larger number
        let sym = ['0', '1']
        delt <- randomDelta sym sts sts
        strt <- elements sts
        accraw <-  sublistOf sts
        let acc = nub (0:accraw)
        return $ DFA sts sym delt strt acc
```

4 NFA Implementation

This section describes our implementation of NFAs. NFAs can sometimes be used over DFAs to more compactly represent a language. However, because we also have implemented ϵ-NFAs, we have chosen to not write any special functionalities for NFAs.

4.1 Data type and basics

In design, the only (natural) difference to DFAs is that the transition function maps to a list instead of to a single state, making it behave like a relation. The `show` instance of NFAs is similar to that of DFAs as well.

```
{-# LANGUAGE FlexibleInstances #-}
module NFA where
import Data.List ( nub, union )
import DFA ( Symbol )
import Test.QuickCheck
    ( elements, sublistOf, Arbitrary(arbitrary), Gen )

data NFA a= NFA { statesNF :: [a]
                , alphabetNF:: [Symbol]
                , deltaNF :: Symbol -> a -> [a]
                , startNF:: a
                , acceptstateNF:: [a]}
```

```
instance Show a => Show (NFA a) where
    show (NFA sts alph del strt acc)
      = "NFA" ++ "("++ show sts ++ "," ++ show alph ++ "," ++ show1 del ++ "," ++ show strt ++
           "," ++ show acc ++")" where
      show1 f = show ["d" ++ "(" ++ show sym ++ "," ++ show st ++ ")" ++ " = " ++ show (f sym
          st) | sym <- alph, st <- sts ]
```

We also implement a similar, but more complicated evaluation function suitable for NFAs. We have to keep track of all possible states we can be in via a helper function `stateArrNF'`. We define the `unionL` helper function that takes the set union of elements of a list of lists. We make use of this functions on several other occasion throughout the code.

```
unionL:: Eq a => [[a]] -> [a]
unionL = foldr union []

evaluateNF:: Eq a => NFA a-> String -> Bool
evaluateNF nf st = all (`elem` alphabetNF nf) st && -- captures elements of string in alphabet
             any (`elem` acceptstateNF nf) (stateArrNF' [startNF nf] st)  where
                stateArrNF'  qs [] = qs -- type signature: [a]->Symbol-> [a]
                stateArrNF'  [] _ = []
                stateArrNF'  qs (x:xs) = unionL [stateArrNF' (deltaNF nf x q) xs | q <- qs] --
                    recursion on string
```

We implemented the NFA-instance for `Arbitrary` by slightly modifying the relevant code for DFAs. The difference is that the arbitrary transition functions we generate here output lists as values.

```
-- recursively make a valuation function for these worlds:
randomFunFromTolist :: (Eq a, Arbitrary a) => [a] -> [a] -> Gen (a -> [a])
randomFunFromTolist [] _ = return (const undefined)
randomFunFromTolist (w:ws) ps = do
    f <- randomFunFromTolist ws ps
    wResult <- sublistOf ps
    return $ \v -> if v == w then wResult else f v

randomDeltaNF :: (Eq a, Arbitrary a) => [Symbol] -> [a] -> [a] -> Gen (Symbol -> a -> [a])
randomDeltaNF [] _ _ = return (const undefined)
randomDeltaNF (sym:syms) ds ps = do
    f <- randomDeltaNF syms ds ps
    wResult <- randomFunFromTolist ds ps
    return $ \sy -> if sy == sym then wResult else f sy

instance Arbitrary (NFA Int) where
    arbitrary = do
        -- choose a set of up to 6 worlds:
        sts <- (0 :) <$> sublistOf [1..5]
        let sym = ['0', '1']
        delt <- randomDeltaNF sym sts sts
        strt <- elements sts
        accraw <- sublistOf sts
        let acc = nub (0:accraw)
        return $ NFA sts sym delt strt acc
```

5 Implementation of ϵ-NFAs

Here we modify our implementation of NFAs to account for epsilon transitions. Even though for every ϵ-NFA there is an equivalent NFA, it is useful to have these transitions for representational purposes of some automata, and for understandability of some constructions.

5.1 Data type and basics

We chose to represent ϵ transitions via a list of tuples of ϵ-related states. This is for convenience when taking the ϵ-closure and to highlight the different role ϵ-transitions have compared to standard transitions. The `show` instance is similar to its DFA counterpart, with the ϵ-relations also listed.

```
{-# LANGUAGE FlexibleInstances #-}
module ENFA where
```

```
import Data.List ( nub, sort )
import DFA ( Symbol )
import NFA ( randomDeltaNF, unionL )
import Test.QuickCheck ( elements, sublistOf, Arbitrary(arbitrary) )
data ENFA a = ENFA { statesENF :: [a]
                   , alphabetENF:: [Symbol]
                   , deltaENF :: Symbol -> a -> [a]
                   , epTrans:: [(a, a)]
                   , startENF:: a
                   , acceptstateENF:: [a]}

instance Show a => Show (ENFA a) where
    show (ENFA sts alph del eps strt acc)
      = "ENFA" ++ "("++ show sts ++ "," ++ show alph ++ "," ++ show1 del ++ "," ++ show eps ++
           "," ++ show strt ++ "," ++ show acc ++")" where
      show1 f = show ["d" ++ "(" ++ show sym ++ "," ++ show st ++ ")" ++ " = " ++ show (f sym
          st) | sym <- alph, st <- sts ]
```

To define ϵ-closure, we need some machinery to calculate the reflexive and transitive closure of some relation. This work is inspired by [Jel13].

```
trClose :: Eq a => [(a, a)] -> [(a, a)]
trClose closure
  | closure == closureUntilNow = closure
  | otherwise                  = trClose closureUntilNow
  where closureUntilNow =
          nub $ closure ++ [(a, c) | (a, b) <- closure, (b', c) <- closure, b == b']

refClose:: Eq a => [a] -> [(a,a)] -> [(a,a)]
refClose as ps = nub (ps ++ [(x,x) | x <- as])

rtClose:: (Eq a, Ord a) => ENFA a -> [a] -> [a]
rtClose nf qs = sort (unionL [[p | p <- statesENF nf , (q, p) `elem` trClose rcl ] | q<- qs] )
     where
                                          rcl = refClose (statesENF nf) (epTrans nf)
```

We use the above to modify the evaluation function to account for ϵ-transitions.

```
evaluateENF:: (Eq a, Ord a) => ENFA a -> String -> Bool
evaluateENF nf st = all (`elem` alphabetENF nf) st && -- captures elements of string in
    alphabet
             any (`elem` acceptstateENF nf) (stateArrENF' [startENF nf] st)  where
                stateArrENF' qs [] = rtClose nf qs
                stateArrENF' [] _ = []
                stateArrENF' qs (x:xs)  =
                 unionL ( [rtClose nf (stateArrENF' (unionL (map (deltaENF nf x) (rtClose nf [
                     q])))) xs ) | q <-qs ])
  -- this first does transitive closure on the input list, then transitive closure on output
      list and takes union
```

5.2 Arbitrary Generation

We make ENFAs instance of `Arbitrary` by slightly modifying the relevant code for NFAs, to the effect of adding an arbitrary ϵ-relation list.

```
instance Arbitrary (ENFA Int) where
    arbitrary = do
        -- choose a set of up to 10 worlds:
        sts <- (0 :) <$> sublistOf [1..5]
        let sym = ['0', '1']
        delt <- randomDeltaNF sym sts sts
        eps <- sublistOf [(x, y) | x <- sts, y <- sts]
        strt <- elements sts
        accraw <- sublistOf sts
        let acc = nub (0:accraw)
        return $ ENFA sts sym delt eps strt acc
```

6 Regular expressions

In this section, we discuss the implementation of regular expressions, as well as a small toolbox to manipulate them.

6.1 Data type and basics

Recall that a regular expression is defined to be either empty, the emptystring, a single symbol, the union or concatenation of two expressions, or the closure of a language (with or without the emptystring). The datatype `RegExp` we used closely follows this definition, but also allows for some more flexibility.

```
module RegExp where
import DFA
import Test.QuickCheck

data RegExp = Empty | Epsilon | R [Symbol] | Union RegExp RegExp | Con RegExp RegExp | Star
    RegExp | Plus RegExp
  deriving (Show, Eq)
```

The main difference between `RegExp` and regular expressions, is in the base case for single symbols in the original definition. We chose to allow a list of symbol to increase user-friendliness. This means that instead of `Con (R "a") (R "b")`, one can write `R "ab"` to denote the same expression. `R []` is also equivalent to `Epsilon`.
For convenience, we also added a function that takes a list of regular expressions and outputs the expression corresponding to the union of all its members.

```
regExpUnion :: [RegExp] -> RegExp
regExpUnion = simplify . foldr Union Empty
```

6.2 Pretty printing and simplifying

As larger expressions tend to become unreadable, we implemented a function for pretty showing and pretty printing regular expressions.

```
pRegExp :: RegExp -> String
pRegExp Empty = ""
pRegExp Epsilon = "R e"
pRegExp (R []) = "R e"
pRegExp (R xs) = show xs
pRegExp (Union r1 r2) = "(" ++ pRegExp r1 ++ "|" ++ pRegExp r2 ++ ")"
pRegExp (Star r) = "(" ++  pRegExp r ++ ")*"
pRegExp (Con r1 r2) = pRegExp r1 ++ pRegExp r2
pRegExp (Plus r) = "(" ++ pRegExp r ++ ")+"

ppRegExp :: RegExp -> IO ()
ppRegExp = print . pRegExp
```

Here is an example to illustrate the improvement on the readability.
`ppRegExp (Con (Union (Con (R "5") (R"3")) (R [])) (Star (R "19")))` yields `"("53"|R e)("19")*"`.
Another way to improve readability is to simplify regular expressions. With simplifying we mean to remove unnecessary parts of an expression. E.g. any expression `r` concatenated with `Empty` is equivalent to `Empty`, and `Con r Empty` and `Union r Epsilon` are equivalent to just `r`. The function `simplify` takes a `RegExp` and outputs an equivalent one with less clutter. As minimising a regular expression is computationally demanding [GS07], this function only does basic simplifications, but runs in polynomial time.

```
simplify :: RegExp -> RegExp
simplify r | r == simplify' r = r
 | otherwise = simplify $ simplify' r where
  simplify' Empty = Empty
```

```
  simplify' Epsilon = Epsilon
  simplify' (R xs) = R xs
  simplify' (Con Epsilon Epsilon) = Epsilon
  simplify' (Con _ Empty) = Empty
  simplify' (Con Empty _) = Empty
  simplify' (Con r' Epsilon) = simplify' r'
  simplify' (Con Epsilon r') = simplify' r'
  simplify' (Con r1 r2) = Con (simplify' r1) (simplify' r2)
  simplify' (Union Empty r') = simplify' r'
  simplify' (Union r' Empty) = simplify' r'
  simplify' (Union r1 r2) | r1 /= r2 = Union (simplify' r1) (simplify' r2)
                          | otherwise = simplify' r1
  simplify' (Plus Epsilon) = Epsilon
  simplify' (Star Epsilon) = Epsilon
  simplify' (Plus Empty) = Empty
  simplify' (Star Empty) = Epsilon
  simplify' (Star r') = Star (simplify' r')
  simplify' (Plus r') = Plus (simplify' r')
```

6.3 Arbitrary generation of regular expressions

We create an `Arbitrary` instance for `RegExp` in order to generate regular expressions. The main purpose for this is testing of properties. For this reason we exclude the `Empty` expression from the generation, as empty expressions cannot generate words. The parameter for `sized` decreases rapidly in order to keep the generated expressions of a reasonable and readable size. This is necessary for feasible runtimes in the translation process from expressions to DFAs later on.

```
instance Arbitrary RegExp where
  arbitrary = sized randomReg where
    randomReg :: Int -> Gen RegExp
    randomReg 0 = R <$> elements ["0", "1"]
    randomReg n = oneof [ R <$> elements ["0", "1"]
                        , Star <$> randomReg (n `div` 8)
                        , Plus <$> randomReg (n `div` 8)
                        , Con <$> randomReg (n `div` 8)
                        <*> randomReg (n `div` 8)
                        , Union <$> randomReg (n `div` 8)
                        <*> randomReg (n `div` 8)
                        , return Epsilon ]
```

6.4 Generating words from regular expressions

The following code is used to generate words from expressions. The intended use is to combine it with `quickCheck` to generate the strings. Note that an `error` is thrown when generating from the `Empty` expression. This is because it is not possible to generate words from nothing. Moreover, the output needs to be distinguished from the empty string. To make sure this error is only thrown when explicitly running `generateString Empty` and not when, for example, executing `generateString (Union (R "0") Empty)`, we use `simplify`.

```
generateString :: RegExp -> Gen String
generateString = (`generateString'` 5) . simplify
  where generateString' :: RegExp -> Int -> Gen String
        generateString' Empty _ = error "cannot generate from empty language"
        generateString' Epsilon _ = return ""
        generateString' (R xs) _ = return xs
        generateString' (Union r1 r2) n = oneof [generateString' r1 n, generateString' r2 n]
        generateString' (Con r1 r2) n = do
            w <- generateString' r1 n
            v <- generateString' r2 n
            return (w ++ v)
        generateString' (Star _) 0 = return ""
        generateString' (Star r) n = oneof [generateString' Epsilon n, generateString' (Con r
            (Star r)) (n - 1)]
        generateString' (Plus r) n = generateString' (Con r (Star r)) n
```

7 Translation functions

Here we implement all the algorithms that translate between the different FSAs and regular expressions. For each instance, we give a short theoretical background and then discuss our implementation.

```
module Translation where
import Data.List
import NFA
import DFA
import ENFA
import RegExp
import Data.Maybe (fromJust)
```

7.1 The Powerset construction

As stated earlier, (ϵ)-NFAs and DFA-s are equivalent in their computing power. This fact is shown by constructing an equivalent DFA from an (ϵ) - NFA. This procedure is called the powerset construction, which we detail here.

Definition 7.1. (Powerset construction - NFA-s) Given an NFA $N = (Q_n, \Sigma, \delta_n, q_0, F_n)$, its powerset construct is a DFA $D = (\mathcal{P}(Q_n), \Sigma, \delta, S, F_D)$, where

- $\mathcal{P}(Q_n)$ is the powerset of Q_n,
- $S = \{q_0\}$
- $F_D = \{R \subseteq Q_N \mid R \cap F_n \neq \emptyset\}$
- $\delta_D(R, \sigma) = \cup_{s \in R} \delta_N(s, \sigma)$

For ϵ-NFA-s the powerset construct is defined similarly, with the distinction that $S = \text{ecl}(\{q_0\})$ and $\delta_D(R, \sigma) = \text{ecl}(\cup_{s \in R} \delta_N(s, \sigma))$. Here, $\text{ecl}(A)$ stands for the set of states reachable from A via ϵ-transitions.

Theorem 1. *An (ϵ)-NFA N and its powerset construct are equivalent.*

We implement the powerset construction for NFA-s. The states of the powerset construct are sublists of states of the NFA. Note that the standard powerset construction is greedy: it introduces exponentially many states. For running purposes, we cut the resulting DFA by our cutDFA function. In some examples, this could cut down a DFA on 2^{19} states to as few as 10.

```
nfaToDFA:: (Eq a, Ord a) => NFA a -> DFA [a]
nfaToDFA (NFA sts alph del strt ac) =
  cutDFA $ DFA sortedStates alph del' (sort [strt]) [st | st <- sortedStates,  intersect st ac
       /= []] where
    del' sy ls = unionL [del sy l | l <- ls]
    sortedStates = map sort $ subsequences sts
```

For ϵ-NFA-s, we need to take care to take ϵ-closures at each relevant step.

```
enfaToDFA:: (Eq a, Ord a) => ENFA a -> DFA [a]
enfaToDFA (ENFA sts alph del eps strt ac) = let nf = ENFA sts alph del eps strt ac in
  cutDFA $ DFA sortedStates alph del' (rtClose nf (sort [strt])) [st | st <- sortedStates,
      intersect st ac /= []] where
    del' sy ls = let nf = ENFA sts alph del eps strt ac in rtClose nf ( unionL [del sy l | l
        <- ls] )
    sortedStates = map sort $ subsequences sts
```

7.2 From regular expressions to ϵ-NFAs

The translation from regular expressions to ϵ-NFAs we implemented is done by recursion on the structure of regular expressions. First we define automata for the basic forms of regular expressions: $\emptyset$, ϵ and the list of symbols. Then for the complex expressions, we combine the automata of its components accordingly. This way we get an automaton that exactly recognizes the language generated by the regular expression.

Theorem 2. *Let r be any regular expression. Then there exists an ϵ-NFA N such that $\mathcal{L}(r) = \mathcal{L}(N)$.*

Proof. Let r be any regular expression on any alphabet Σ. We use structural induction on r.

- For $r := \emptyset$, consider ϵ-NFA $N = (\{q\}, \Sigma, \emptyset, \emptyset, q, \emptyset)$. Recall that $\mathcal{L}(r) = \mathit{emptyset}$. N rejects on every input as no path can end up in an accepting state, because there are no accepting states. Hence $\mathcal{L}(N) = \emptyset$.
- For $r := \epsilon$, recall that $\mathcal{L}(r) = \{\epsilon\}$. Now consider ϵ-NFA $N = (\{q\}, \emptyset, \emptyset, \emptyset, q, \{q\})$. On the empty input, N ends in the start state, which is also an accepting state, so N accepts. On any other input, N rejects as no outgoing arrows are defined for the starting state. Hence $\mathcal{L}(N) = \{\epsilon\}$.
- For $r := a$ with $a \in \Sigma$, recall that $\mathcal{L}(r) = \{a\}$. Construct $N = (\{s, t\}, \Sigma, \{(s, a, t\}, \emptyset, s, \{t\})$. Observe that N rejects the empty string, as the initial state is not accepting. Because there is just one transition, if N reads anything other than a, it will reject. If N reads a and anything after that, it will also reject. If it just reads a and nothing more, N will accept as t is accepting. Hence $\mathcal{L}(N) = \{a\}$.

Now suppose for the induction hypothesis *(IH)* that there are automata $N_1 = (S_1, \Sigma_1, \Delta_1, \epsilon_1, s_1, F_1)$ and $N_2 = (S_2, \Sigma_2, \Delta_2, \epsilon_2, s_2, F_2)$, such that $\mathcal{L}(N_i) = \mathcal{L}(r_i)$ for $i \in \{1, 2\}$. For reasons of space we will only discuss the union extensively. The full proof can be found in [Sip13].

- Let $r := r_1 + r_2$. Now construct $N = (S_1 \uplus S_2 \uplus \{s_0\}, \Sigma_1 \cup \Sigma_2, \Delta_1 \uplus \Delta_2, \epsilon_1 \uplus \epsilon_2 \cup \{(s_0, s_1), (s_0, s_2)\}, s_0, F_1 \uplus F_2)$, where s_0 is a fresh state not in $S_1 \uplus S_2$ and $\uplus$ denotes the disjoint union. Let $w \in \mathcal{L}(r_1) \cup \mathcal{L}(r_2)$. Then $w \in \mathcal{L}(r_i)$ for some $i \in \{1, 2\}$. Hence by *IH*, w gets accepted by N_i, i.e. there is a run of w that starts in s_i and ends in some accepting state f in F_i. But then there is also a run of w on N such that it ends in f, as there is an epsilon transition from s_0 to s_i. Hence N also accepts. Conversely, if $w \notin \mathcal{L}(r_1) \cup \mathcal{L}(r_2)$, then both N_1 and N_2 have no successful runs on w, so it is easy to see that N will also not have a successful run and thus rejects.
- For $r = r_1 r_2$, construct N by taking the disjoint union of the states and transitions of N_1 and N_2. Let s_1 be the starting state, let F_2 be the accepting states. The crux is to add an epsilon arrow from every state in F_1 to s_2, ensuring that exactly the concatenation is recognised.
- For $r = r_1^*$, Let N be N_1, but with additional ϵ-arrows from the accepting states to the start state.
- For $r = r_1^+$ it is enough to observe that r_1^+ is equivalent to the concatenation of r_1 with r_1^*.

□

Our implementation of the translation follows the idea of the above proof. To implement taking the disjoint union, we first defined a function that takes two automata and outputs an automaton equivalent to the second argument, but disjoint from the first.

```
-- function that takes two ENFAs and outputs a relabeling of the second ENFA such that the
    states of both become disjoint
makeDisjoint :: ENFA Int -> ENFA Int -> ENFA Int
makeDisjoint n1 n2 = ENFA states' alphabet' delta' epT start' accept where
  add | minimum (statesENF n2) <= 0 = negate (minimum (statesENF n2)) + (maximum (statesENF n1
       ++ statesENF n2) + 1)
      | otherwise = maximum (statesENF n1 ++ statesENF n2) + 1
  states' = map (+ add) (statesENF n2)
  alphabet' = alphabetENF n2
  delta' sym state = map (+ add) (deltaENF n2 sym (state - add))
  epT = [(s + add, t + add) | (s,t) <- epTrans n2 ]
  start' = startENF n2 + add
  accept = map (+ add) (acceptstateENF n2)
```

With this function, we can safely take unions between and concatenate automata.

```
unionENFA :: ENFA Int -> ENFA Int -> ENFA Int-- Use only if states are disjoint
unionENFA n1 n2 = ENFA states'' alphabet2 delta'' epT start'' accept where
  states'' = start'' : statesENF n1 ++ statesENF n2
  alphabet2 = alphabetENF n1 `union` alphabetENF n2
  delta'' sym st = deltaENF n1 sym st ++ deltaENF n2 sym st
  epT = epTrans n1 ++ epTrans n2 ++ [(start'', startENF n1), (start'', startENF n2)]
  start'' = maximum (statesENF n2 ++ statesENF n1) + 1
  accept = acceptstateENF n1 ++ acceptstateENF n2

concatENFA :: ENFA Int -> ENFA Int -> ENFA Int -- Use only if states are disjoint again
concatENFA n1 n2 = ENFA states4 alphabet'' delta3 epT start3 accept where
  states4 = statesENF n1 ++ statesENF n2
  alphabet'' = alphabetENF n1 `union` alphabetENF n2
  delta3 sym st = deltaENF n1 sym st ++ deltaENF n2 sym st
  epT = epTrans n1 ++ epTrans n2 ++ [(s, startENF n2) | s <- acceptstateENF n1]
  start3 = startENF n1
  accept = acceptstateENF n2

starENFA :: ENFA Int -> ENFA Int
starENFA n = ENFA (statesENF n) (alphabetENF n) (deltaENF n) ep (startENF n) (acceptstateENF n
    )  where
  ep = epTrans n ++ [(startENF n, s)| s <- acceptstateENF n] ++ [(s, startENF n) | s <-
      acceptstateENF n]
```

Combining the above then gets us the following function for translating regular expressions to ϵ-NFAs.

```
regExpToENFA :: RegExp -> ENFA Int
regExpToENFA Empty   = ENFA [1] [] (\_ _ -> []) [] 1 []
regExpToENFA Epsilon = ENFA [1] [] (\_ _ -> []) [] 1 [1]
regExpToENFA (R xs)  = ENFA [0..length xs] (nub xs) delta2 [] 0 [length xs] where
  delta2 symbol state | state >= length xs || state < 0 = []
                      | xs !! state == symbol = [state + 1]
                      | otherwise = []
regExpToENFA (Union r1 r2) = regExpToENFA r1 `unionENFA` makeDisjoint (regExpToENFA r1) (
    regExpToENFA r2)
regExpToENFA (Star r) = starENFA (regExpToENFA r)
regExpToENFA (Con r1 r2) = regExpToENFA r1 `concatENFA` makeDisjoint (regExpToENFA r1) (
    regExpToENFA r2)
regExpToENFA (Plus r) = regExpToENFA r `concatENFA` makeDisjoint (regExpToENFA r) (starENFA (
    regExpToENFA r))
```

To go from regular expressions to DFAs, one can simply use the above function and then run the powerset construction on the result.

7.3 From DFAs to regular expressions

It is not immediately obvious that every DFA is also equivalent to some regular expression. However, there are several ways to convert DFAs into equivalent regular expressions. We will explain one way we also implemented here. The method is taken from [Chi18]. There you can also find the full proof of the following theorem.

Theorem 3. *For every DFA D, there is an equivalent regular expression r such that $\mathcal{L}(D) = \mathcal{L}(r)$.*

Proof. Let $D = (S, \Sigma, \delta, s, F)$ be a DFA with $S = \{1, \ldots, |S|\}$.
Next, we are going to define regular expressions R^k_{ij} inductively for all $i, j \in S,\ k \leq |S|$. These expressions are going to correspond to all possible paths from i to j, passing through intermediate no larger than k. For each pair of states i, j, let $A^{ij} := \{a \mid \delta(i, a) = j\}$ be the set of labels on the transitions from i to j.

$$R^0_{ij} = \begin{cases} \bigcup A^{ij} & \text{if } i \neq j \\ \epsilon \cup \bigcup A^{ij} & \text{if } i = j \end{cases}$$

R^0_{ij} thus captures just the labels of direct transitions from i to j, or if $i = j$, also ϵ. For $k > 0$, $R^k_{ij} := R^{k-1}_{ij} \cup (R^{k-1}_{ik}(R^{k-1}_{kk})^* R^{k-1}_{kj})$. This has the following meaning: a path from i to j passing through states no larger than k can either:

- Not pass through k at all. Then this path is covered by the regular expression R^{k-1}_{ij}, which is the first part of the union of R^k_{ij}.
- Pass through k at least once. Then the path contains 3 segments.
  1. The first segment goes from i to k. The expression for these paths are covered by R^{k-1}_{ik}.
  2. The second part loops from k to k, possibly 0 or many times. In any case, the expression is covered by $(R^{k-1}_{kk})^*$.
  3. The last segment goes from k to j. This part is covered by R^{k-1}_{kj}.

  Hence the whole path is covered by the concatenation of those three segments, which is exactly the second part of R^k_{ij}.

Now let n be the starting state of D. The regular expression covering all paths from n to accepting states is thus $\bigcup\{R^{|S|}_{nf} \mid f \in F\}$. □

Our implementation of the translation follows the method of the above proof, with two extra steps. As the expressions output by the procedure can become quite large, we first minimize the DFA to an equivalent minimal DFA. After the whole procedure we use the `simplify` function again, which was defined in the section on regular expressions.
We use an implementation of Brzozowski's algorithm for minimising DFA's [Brz62]. We state without proof that the result of first reversing[2], then determinising and trimming an automaton twice yields an equivalent minimal automaton.

Proposition 4. *Let $D = (S, \Sigma, \delta, s, F)$ be a DFA. Then there is an equivalent minimal DFA D', which is obtained by applying the following proces twice:*

- *reverse the automaton,*
- *determinise the result with the powerset construction, trimming all unreachable states in the process.*

The idea behind the algorithm is as follows. There can be three types of redundancy in an automaton.

1. States that cannot reach the goal.
2. States that cannot be reached by the starting state.

[2]By reversing we mean that arrows from s to t become arrows from t to s, acceptstates become starting states and vice versa. In the case of multiple accepting states, we transform to an ϵ-NFA with one accepting state first, in order to end up with one start state in the reversed automaton.

3. States that are indistinguishable from eachother.

The first type is covered by reversing and then trimming. The second is covered by the second trimming. The third is a result of the application of the power set construction: if states s and t are indistinguishable, then the power set construction will ensure that they end up in the same power states.
In the code we also apply `cutDFA` before the process, because the implementation of the power set construction is not optimised. We furthermore use a small variation of the original construction, by ensuring that the newly added single accepting state will not show up in the resulting DFA.

```
minimizeDFA :: Ord a => DFA a -> DFA [Int]
minimizeDFA = reverseDFA . reverseDFA

reverseDFA :: Ord a => DFA a -> DFA [Int]
reverseDFA d = enfaToDFA' $ ENFA sts alph delt ep st acc where
  e = singleAccept $ cutDFA d
  sts = statesENF e
  alph = alphabetENF e
  delt sym s = [t | t <- statesENF e, s `elem` deltaENF e sym t]
  ep = [(s, t) | (t,s) <- epTrans e ]
  st = head $ acceptstateENF e
  acc = [startENF e]
  enfaToDFA' nf@(ENFA sts' alph' del eps strt ac) =
    cutDFA $ DFA sortedStates alph' del' (rtClose nf (sort [ t | (s, t) <- eps, s == strt])) [
        st' | st' <- sortedStates,  intersect st' ac /= []] where
      del' sy ls = rtClose nf ( unionL [del sy l | l <- ls] )
      sortedStates = map sort $ subsequences sts'

singleAccept :: Ord a => DFA a -> ENFA Int
singleAccept d = ENFA sts alph delt ep st acc where
  d' = makeIntDFA d
  newAccept = maximum (states d') + 1
  sts = newAccept : states d'
  alph = alphabet d'
  delt sym t | t == newAccept = []
             | otherwise =  [delta d' sym t]
  ep = [(f, newAccept) | f <- acceptstate d']
  st = start d'
  acc = [newAccept]
```

The renaming of the states is done in the following way:

```
makeIntDFA :: (Eq a, Ord a) => DFA a -> DFA Int
makeIntDFA dfa = DFA sts alph delt strt acceptst
  where sts = [1..length (states dfa)]
        alph = alphabet dfa
        delt sym i  = indX $ delta dfa sym (states dfa !! (i-1))
        strt = indX (start dfa)
        acceptst = [indX s | s <- acceptstate dfa]
        indX s = fromJust (elemIndex s (states dfa)) + 1
```

Putting everything together, the rest of the algorithm is found here.

```
-- simplify to make the result a bit more readable
dfaToRegExp :: Ord a => DFA a -> RegExp
dfaToRegExp dfa = simplify $ regExpUnion [rijk dfaInt (start dfaInt) f (length $ states dfaInt
    ) | f <- acceptstate dfaInt ]
  where dfaInt = (makeIntDFA . minimizeDFA) dfa

-- rijk dfa outputs the regular expression covering paths from i to j without passing through
    intermediate states larger than k
rijk :: DFA Int -> Int -> Int -> Int -> RegExp
rijk dfa i j 0 | i == j    = simplify $ regExpUnion (Epsilon : labels)
               | otherwise = simplify $ foldr Union Empty labels
  where labels =  [R [x] | x <- alphabet dfa, delta dfa x i == j]
rijk dfa i j k = Union (rijk dfa i j (k-1))
  (Con (rijk dfa i k (k-1))
  (Con (Star $ rijk dfa k k (k-1))
  (rijk dfa k j (k-1))))
```

One nice property of minimal DFAs is that they are unique for each language (up to isomorphism). This transforms checking whether two DFAs have the same language to a graph isomorphism

problem on the minimal equivalents of said DFAs. That is done by the following function.

```
-- Checks whether two DFAs are equivalent
brzozowski :: (Ord a, Ord b) => DFA a -> DFA b -> Bool
brzozowski d d' = alphabet d == alphabet d' &&
  (null (acceptstate m) && null (acceptstate m')) ||
   isIsomorphTo (start m) (start m') [] where
    m = minimizeDFA d
    m' = minimizeDFA d'
    isIsomorphTo s s' is =
      (s,s') `elem` is || (s `elem` acceptstate m) == (s' `elem` acceptstate m') &&
      all (\sym -> isIsomorphTo (delta m sym s) (delta m' sym s') ((s,s'):is)) (alphabet m)
```

8 Tests

This section discusses our testing suite. We import all modules and test different aspects of the code via several `QuickCheck` queries.

```
{-# LANGUAGE ScopedTypeVariables #-}
module Main where
import Test.QuickCheck
import RegExp
import DFA
import ENFA
import NFA
import Translation
```

The main function runs all of our tests and prints a one-line explanation for them.

```
main :: IO()
main = do
    print "ZeroStart accepts words starting with 0, for DFA, NFA and ENFA version:"
    test1FA
    print "ZeroStart does not accept other words, for DFA, NFA and ENFA version:"
    test2FA
    print "The reverse of the reverse accepts and rejects the same words:"
    test1ReverseDFA
    print "The reverse of the reverse is equivalent to the original:"
    test2ReverseDFA
    print "Minimize yields an automaton that accepts and rejects the same words:"
    test1MinimizeDFA
    print "Minimize doesn't yield more states:"
    test2MinimizeDFA
    print "cutDFA DFA accepts and rejects the same as DFA:"
    test1CutDFA
    print "Powerset construction yields equivalent DFA for NFA:"
    test1PowNF
    print "Powerset construction yields equivalent DFA for ENFA:"
    test1PowENF
    print "DFA accepts words generated by the corresponding RegExp:"
    test1DFtoReg
    print "DFA does not accept words generated by the RegExp corresponding to the complement
        DFA:"
    test2DFtoReg
    print "ENFA corresponding to RegExp accepts words generated from RegExp:"
    test1RegtoENF
    print "Composition of translations yields equivalent DFA for DFA:"
    test1DFeqv
    print "Complement RegExps don't generate the same word:"
    test1CompRegx
    print "Complement on different RegExp:"
    test2CompRegx
    print "Minimised automata generated from a RegExp accept words from said RegExp:"
    testMinimizeMore
```

First we start by defining some example NFAs and ϵ-NFAs similar to the previously seen DFA called `zeroStart`, that accept the same language. We also add a further example for a DFA.

```
zeroStartNF:: NFA Int
zeroStartNF = NFA [0,1,2, 3] ['0', '1'] deltazeroNF 0 [1] where
    deltazeroNF:: Symbol-> Int -> [Int]
    deltazeroNF char st
```

```
      | st == 0 && char == '0' = [1]
      | st == 0 && char == '1' = [2, 3]
      | st == 1 = [1]
      | st == 2 = [2, 3]
      | st == 3 = [3]
      | otherwise = [-1] --added to avoid hlint warning 'pattern matches are non-exhaustive'

zeroStartENF:: ENFA Int
zeroStartENF = ENFA [0,1,2, 3] ['0', '1'] deltazeroNF [(3,2), (2,3)] 0 [1] where
    deltazeroNF:: Symbol-> Int -> [Int]
    deltazeroNF char st
      | st == 0 && char == '0' = [1]
      | st == 0 && char == '1' = [2, 3]
      | st == 1 = [1]
      | st == 2 = [2, 3]
      | st == 3 = [3]
      | otherwise = [-1]

dfa2:: DFA Int
dfa2 = DFA [0,1,3] "01" del2 3 [0] where
    del2 char st
      | char =='0' && st == 0 = 3
      | char =='0' && st == 1 = 0
      | char =='0' && st == 3 = 3
      | char =='1' && st == 0 = 1
      | char =='1' && st == 1 = 3
      | char =='1' && st == 3 = 3
      | otherwise = -1
```

Note the regular expression for the language of words starting with 0 is $0(0+1)^*$. We generate words from this expression and test if all 3 of our example automata for this language accept the generated words. Next, we generate words from the complement language, which has the regular expression $\epsilon + 1(0+1)^*$ and test if the automata reject these words.

```
test1FA :: IO ()
test1FA = quickCheck (forAll (generateString (Con (R "0") (Star (Union (R "0") (R "1")))))
                        (\w -> zeroStart `evaluate` w && zeroStartNF `evaluateNF` w  &&
                        zeroStartENF `evaluateENF` w ))
test2FA :: IO ()
test2FA = quickCheck (forAll (generateString (Union Epsilon (Con (R "1") (Star (Union (R "0")
    (R "1"))))))
                        (\w -> not (zeroStart `evaluate` w) && not (zeroStartNF `evaluateNF` w
                            )  &&  not(
                        zeroStartENF `evaluateENF` w )))
```

Here we test the toolkit for obtaining the minimal DFA for a language. We test if the reverse of the reverse DFA accept the same words as the original DFA. We go further and use the `brozowski` function to test whether they hare equivalent. We then test if a DFA and its corresponding minimal DFA accept the same words. We finish by testing that the minimal DFA has always at most as many states as the original DFA. We also test the `cutDFA` function. We check if the DFA after cutting accepts the same words as the original DFA. We generate arbitrary DFAs for these.

```
test1ReverseDFA :: IO ()
test1ReverseDFA = quickCheck (\r (d:: DFA Int)  -> forAll (generateString r) (\w -> d `
    evaluate` w == (reverseDFA . reverseDFA ) d `evaluate` w ))
test2ReverseDFA :: IO ()
test2ReverseDFA = quickCheck (\(d :: DFA Int) -> brzozowski d ((reverseDFA . reverseDFA) d) )
test1MinimizeDFA :: IO ()
test1MinimizeDFA = quickCheck (\r (d :: DFA Int) -> forAll (generateString r) (\w -> d `
    evaluate` w == minimizeDFA d `evaluate` w ))
test2MinimizeDFA :: IO ()
test2MinimizeDFA = quickCheck (\ (d :: DFA Int) -> length (states d) >= length (states (
    minimizeDFA d)))
test1CutDFA :: IO ()
test1CutDFA = quickCheck (\r (d :: DFA Int) -> forAll (generateString r) (\w -> d `evaluate` w
     == cutDFA d `evaluate` w ))
```

We now test the powerset construction. We test if an NFA accepts the same binary strings as its powerset construct. We repeat for ϵ-NFAs. We generate arbitrary NFAs and ϵ-NFAs for this.

```
test1PowNF :: IO ()
```

```
test1PowNF = quickCheck (forAll (generateString (Union (R "0") (R "1")))
                            (\w (enf:: NFA Int) -> evaluateNF enf w == evaluate (nfaToDFA enf)
                                 w) )

test1PowENF :: IO ()
test1PowENF = quickCheck (forAll (generateString (Union (R "0") (R "1")))
                            (\w (enf:: ENFA Int) -> evaluateENF enf w == evaluate (enfaToDFA
                                enf) w) )
```

We now test the translation from DFA to `RegExp`. We check whether a DFA accepts words generated from the corresponding regular expression and that it does not accept words generated from the regular expression corresponding to the DFA with complement accept states. We generate arbitrary DFAs for this.

```
test1DFtoReg :: IO ()
test1DFtoReg = quickCheck (\ (df:: DFA Int) -> if dfaToRegExp df /= Empty then
                forAll (generateString (dfaToRegExp df)) (\w -> df `evaluate` w) else property
                     True)

test2DFtoReg :: IO ()
test2DFtoReg = quickCheck ( \ (df:: DFA Int) -> if dfaToRegExp (flipDFA df) /= Empty then
                forAll (generateString (dfaToRegExp (flipDFA df))) (not . evaluate df ) else
                    property True)
```

We now test the translation from `RegExp` to ϵ-NFA. We check whether the ϵ-NFA corresponding to a regular expression accepts words generated from the expression. We generate arbitrary regular expressions for this.

```
test1RegtoENF :: IO ()
test1RegtoENF = quickCheck ( \ (reg:: RegExp) -> forAll (generateString reg)
                        (\w -> regExpToENFA reg `evaluateENF` w))
```

We put everything together and test whether a DFA and the DFA obtained from translating to regular expressions, then to ϵ-NFA and back to DFA are equivalent. We test it on our two example DFA-s. We also calculate complement regular expressions by translating to DFAs, taking the complement of accept states and translating back to regular expressions. We test such complement expressions don't yield the same words. We test this on 2 toy expressions. We also combine our translations by the minimal DFA toolkit and check that the minimal DFA corresponding to a regular expression accepts the words generated from it.

```
test1DFeqv :: IO ()
test1DFeqv = quickCheck (forAll (generateString (Union (R "0") (R "1")))
                (\w -> evaluate dfa2 w == evaluate (func dfa2) w &&  evaluate zeroStart w ==
                    evaluate (func zeroStart) w)) where
                    func = enfaToDFA . regExpToENFA . dfaToRegExp
test1CompRegx :: IO ()
test1CompRegx = quickCheck (forAll (generateString reg) (\w -> forAll (generateString (creg
    reg))(/= w))) where
                    creg = dfaToRegExp . makeIntDFA . flipDFA . enfaToDFA . regExpToENFA
                    reg = Union Epsilon (Con (R "1") (Star (Union (R "0") (R "1"))))
test2CompRegx :: IO ()
test2CompRegx = quickCheck (forAll (generateString reg) (\w -> forAll (generateString (creg
    reg))(/= w))) where
                    creg = dfaToRegExp . makeIntDFA . flipDFA . enfaToDFA . regExpToENFA
                    reg = Con (Star (R "2")) (Con (R "0") (R "0"))
testMinimizeMore :: IO ()
testMinimizeMore = quickCheck (\r -> forAll (generateString r) (\w -> (minimizeDFA . enfaToDFA
     . regExpToENFA) r `evaluate` w))
```

All tests run in a reasonable amount of time and are passed.

9 Conclusion and future work

In this report we introduced some background theory on formal languages and automata. Then we discussed our implementation of DFAs, (ϵ-)NFAs and regular expressions. Furthermore, we successfully implemented translations between these FSAs and expressions. We designed and

defined test functions to test certain properties of our implementation and of automata and regular expressions in general. All of our implementation seems to behave as expected. We thus created a tool that can be used to gain some understanding about automata and regular languages.
However, the program could be optimised more, and some more features can be added to improve functionality and user-experience.
Regarding optimisations, one major point is the exponential runtime and space that some algorithms now take. Even though taking the power set is exponential by definition, the power set construction can be made more efficient by only ever considering reachable states in the process, not just after. This would not only have a positive impact on the construction itself, but also on the Brzozowski minimisation.
Furthermore, a better algorithm to simplify regular expressions would be useful. Especially because with better simplifications, one can test the program more extensively; constructing a regular expression from a DFA and then calculating the corresponding DFA again now often takes very long.
In the isomorphism checker, the implementation could also be better, as now it is doing a lot of the same work more than once.
Regarding future features, it would be ideal if the transition systems could be output as (the picture of) a graph. This would make it easier to work with the automata corresponding to regular expressions for example. The addition of a parser for these regular expressions would also enhance the user experience.

References

[Brz62] Janusz A. Brzozowski. Canonical regular expressions and minimal state graphs for definite events. *Mathematical Theory of Automata*, 12(6):529–561, 1962.

[Chi18] Maurice Chiodo. Automata and formal languages lecture notes, December 2018. [Online:] `https://www.dpmms.cam.ac.uk/~mcc56/teaching/`.

[GS07] Gregor Gramlich and Georg Schnitger. Minimizing nfa's and regular expressions. *Journal of Computer and System Sciences*, 73(6):908–923, 2007.

[Jel13] Tikhon Jelvis. Transitive closure from a list using haskell. Computer Science Stack Overflow, 2013. [Online:] `https://stackoverflow.com/a/19214140`.

[Sip13] Michael Sipser. *Introduction to the Theory of Computation*. Course Technology, Boston, MA, third edition, 2013.